SORBONNE UNIVERSITÉ
Faculté des Sciences et Ingénierie
Master SESI
Département Informatique

PROJET DMA

PSESI

Rapport Soutenance

21/05/2026

Présenté par :
Yvonne HUANG
Alexander BAKALOV
Ibrahima BALDÉ
Stéphane JI

Encadrants :
Pirouz BAZARGAN-SABET

Année universitaire 2025-2026

Table des matières

1 Introduction **2**
- 1.1 Contexte . . . 2
- 1.2 Objectifs . . . 2

2 Architecture Global du Système **3**
- 2.1 Vue d'ensemble de la communication Processeur(s) $\leftrightarrow$ DMA . . . 3
- 2.2 Choix de conception majeurs . . . 4

3 Conception de l'interface logicielle **4**
- 3.1 Structure de données et gestion des états . . . 5
- 3.2 Implémentation des fonctions de contrôle (API) . . . 5

4 Validation et Analyse des Résultats logicielle **6**
- 4.1 Test 1 : Tests Fonctionnels et Séquentiels . . . 6
- 4.2 Test 2 : Concurrence et intégrité structurelle . . . 7
- 4.3 Test 3 : Multi-threads et requêtes aléatoires . . . 7

5 Architecture et Conception Matérielle (VHDL) **8**
- 5.1 Interconnexions internes et registres de communication . . . 8
- 5.2 Machines à états finis (FSM) du bloc Corps . . . 9
- 5.3 Machines à états finis (FSM) du bloc Interconnect . . . 10
  - 5.3.1 Début . . . 10
  - 5.3.2 Requête Req_Lock . . . 10
  - 5.3.3 Requête Release_lock . . . 11
  - 5.3.4 Requête Get_Job . . . 12
  - 5.3.5 Requête Read . . . 12
  - 5.3.6 Requête Store . . . 13
  - 5.3.7 Requête Done_job . . . 13
- 5.4 Calcul de performance de l'interface logicielle . . . 13
  - 5.4.1 Méthodologie . . . 14
  - 5.4.2 Résultats . . . 14

1 Introduction

1.1 Contexte

Dans l'architecture traditionnelle d'un système informatique, le processeur (CPU) occupe une place centrale. En tant qu'unité principale de traitement, il est chargé d'exécuter les instructions du programme stockées en mémoire vive. Cependant, l'évolution des besoins logiciels modernes impose des manipulations de données de plus en plus massives. Lorsqu'un transfert de données doit être effectué d'une adresse source A vers une adresse de destination B, le CPU est classiquement sollicité pour chaque itération. Ce processus se traduit par une succession de cycles de lecture et d'écriture, souvent réalisés octet par octet. Cette opération, bien que fondamentale, est purement mécanique et répétitive : elle mobilise un nombre important de cycles d'horloge sans exploiter les capacités de calcul logique du processeur. En agissant ainsi, le CPU se retrouve accaparé par des tâches de gestion de flux, ce qui réduit sa disponibilité pour les traitemements d'intructions plus complexes.

Pour remédier à cette utilisation sous-optimale des ressources, on intègre un contrôleur DMA (Direct Memory Access). Ce composant matériel spécialisé est conçu pour prendre en charge le déplacement des données de manière totalement autonome.

Le fonctionnement est le suivant : le processeur délègue la tâche de transfert au DMA en lui fournissant les paramètres nécessaires (adresses et taille du bloc), puis reprend immédiatement l'exécution de ses propres instructions. Le DMA prend alors le contrôle du bus de données pour copier les blocs de mémoire sans intervention constante du processeur. Cette décharge de travail permet d'accroître les performances globales du système : le CPU reste concentré sur les calculs intensifs tandis que les transferts de données s'opèrent en arrière-plan.

1.2 Objectifs

Le but principal de ce projet est la conception d'un système de transfert de données reposant sur un contrôleur DMA (Direct Memory Access) capable de servir plusieurs processeurs de manière concurrente et sécurisée.Pour répondre à cette problématique d'accès partagé et optimiser l'efficacité globale du système, nos objectifs se divisent en deux axes majeurs :

1. Conception d'une interface logicielle

L'abstraction logicielle doit garantir une communication fluide, à faible latence et exempte de conflits entre le système d'exploitation et le module matériel. Cet axe de développement comprend :

- **Modélisation d'une structure de données partagée :** Définition d'une zone mémoire commune faisant office de passerelle unique entre les mondes logiciel et matériel. Elle repose sur un tableau de descripteurs de tâches (*Jobs*) dont l'organisation en listes chaînées est optimisée selon l'état de la requête (`FREE`, `PENDING`, `DONE`)
- **Développement des primitives de contrôle (*Driver*) :**

- `DMA_Job(src, dest, len)` : Implémentation de la fonction système chargée d'extraire un descripteur libre, d'initialiser les paramètres du transfert et de planifier la tâche dans la file d'attente du contrôleur matériel.
- `DMA_Done(ID)` : Implémentation de la fonction d'acquittement permettant à l'OS de valider le succès d'une opération terminée, de libérer les ressources associées et de réintégrer le descripteur dans le pool disponible.

— **Gestion de la concurrence :** Sécurisation des accès simultanés des processeurs et du contrôleur DMA aux listes d'états par l'intégration de mécanismes de verrouillage (*mutex*) afin de prévenir toute corruption de données ou situation de compétition (*race conditions*)

2. Architecture et modélisation matérielle

Sur le plan matériel, l'objectif est de concevoir un module DMA, divisé en trois blocs fonctionnels, s'intégrant proprement sur le bus système :

— **Spécification fonctionelle et comportementale :** établissement des machines à états finis (FSM) régissant le comportement du contrôleur

— **Conception DMA :** Description VHDL de la DMA.

2 Architecture Global du Système

2.1 Vue d'ensemble de la communication Processeur(s) $\leftrightarrow$ DMA

La communication entre les processeurs de l'architecture SoC (*System on Chip*) et le contrôleur DMA ne s'effectue pas par un lien d'interconnexion direct, mais repose sur un paradigme de mémoire partagée et d'observation du bus système. Cette approche permet de découpler l'exécution logicielle du traitement matériel sous-jacent.

Le mécanisme d'échange s'articule autour d'un protocole asynchrone en trois étapes clés :

1. **Soumission (Logiciel) :** Un processeur configure une requête de transfert en écrivant un descripteur de tâche (*Job*) dans une structure de données dédiée située en mémoire (RAM).
2. **Interception et Exécution (Matériel) :** Le bloc *Snoopy* du contrôleur DMA surveille en permanence les transactions du bus. Dès qu'une modification d'état est détectée, il notifie le *Corps* de la DMA. Ce dernier prend alors l'initiative des transferts en ordonnant au bloc *Interconnect* d'exécuter de manière autonome les cycles de lecture et d'écriture requis en RAM.
3. **Acquittement :** Une fois la copie du bloc de données finalisée, le *Corps* de la DMA met à jour le statut du descripteur en mémoire. Le processeur peut alors libérer la ressource de manière sécurisée.

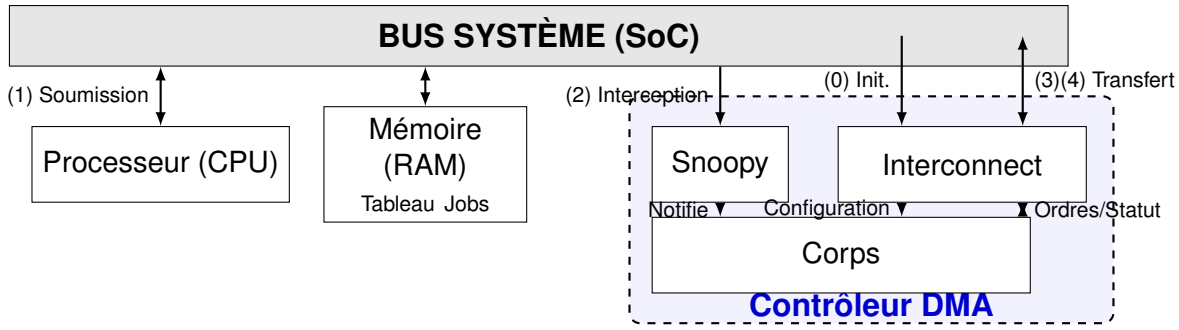


FIGURE 1 – Architecture fonctionnelle matérielle : intégration de l'Interconnect unifié et des flux de contrôle associés

2.2 Choix de conception majeurs

L'efficacité du système reposant sur la rapidité des fonctions logicielles de contrôle, l'architecture doit garantir une complexité algorithmique minimale pour la soumission et l'acquittement des requêtes. Pour répondre à cette contrainte, les choix de conception ont été faits :

- **Communication par mémoire partagée :** L'utilisation d'une structure commune en RAM évite l'engorgement du bus système et permet de mémoriser et centraliser des requêtes provenant de plusieurs sources simultanément.
- **Indexation par listes chaînées :** Plutôt que de parcourir séquentiellement un tableau statique pour identifier les requêtes en attente ou terminées (ce qui imposerait une complexité en $\mathcal{O}(n)$), le système manipule des pointeurs. Les requêtes sont réparties dynamiquement dans des listes distinctes, optimisées selon leur statut opérationnel :
  - **Liste `FREE` (Simplement chaînée) :** Dédiée aux descripteurs disponibles. Elle assure une allocation et une libération instantanées en tête de liste.
  - **Liste `PENDING` (Circulaire) :** Dédiée aux requêtes en attente de traitement matériel. L'organisation circulaire garantit un traitement équitable (*Round-Robin*) par le contrôleur DMA, prévenant ainsi toute famine logicielle.
  - **Liste `DONE` (Doublement chaînée) :** Dédiée aux transferts finalisés. Elle offre une insertion rapide par la DMA et un retrait efficace par le processeur lors de l'appel à la fonction d'acquittement, grâce aux double pointeurs (*Précédent / Suivant*).
- **Verrouillage granulaire (Mutex) :** Afin de permettre une véritable concurrence au sein d'une architecture multi-processeurs sans induire de goulots d'étranglement, chaque liste possède son propre verrou de synchronisation. Cette granularité fine sécurise le passage des descripteurs d'une liste à l'autre tout en minimisant le temps de blocage des processeurs.

3 Conception de l'interface logicielle

L'interface logicielle matérialise l'espace mémoire partagé entre les processeurs et le contrôleur DMA. Cette communication s'appuie sur une structure de données centralisée et un protocole d'états strict géré par le système d'exploitation.

3.1 Structure de données et gestion des états

Le système est architecturé autour d'un tableau statique de descripteurs de taille fixe. Chaque entrée, de type `Job`, contient les variables nécessaires à l'exécution du transfert ainsi que son état courant :

— `FREE` : L'emplacement est disponible dans le pool pour une nouvelle requête.
— `PENDING` : La requête est planifiée et attend son traitement par le contrôleur DMA.
— `DONE` : Le transfert matériel est achevé ; l'emplacement attend l'acquittement du processeur.

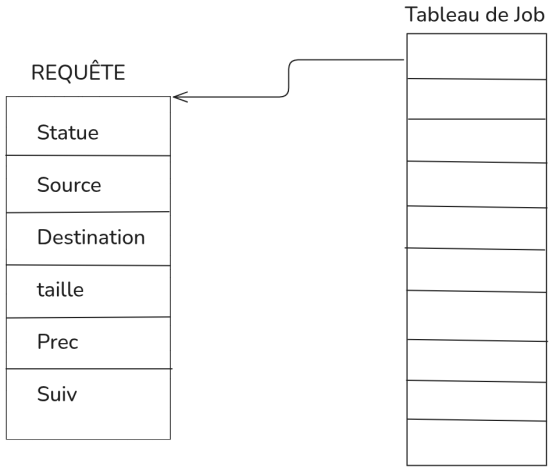


FIGURE 2 – Structure d'un descripteur (`Job`)

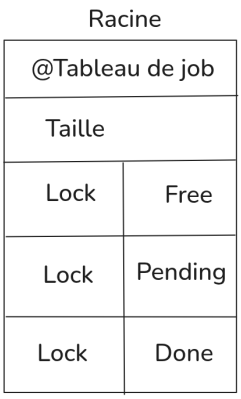


FIGURE 3 – Architecture de la racine

Comme établi lors des choix de conception (Section 2.2), une structure racine organise ces descripteurs en listes chaînées distinctes selon leur état (Tableau 1). Pour permettre un accès concurrent sécurisé entre les processeurs et la DMA, chaque liste est protégée par un verrou matériel dédié (*mutex*).

TABLE 1 – Propriétés des listes d'états de la structure racine

État	**Contrainte opérationnelle**	**Structure implémentée**
`FREE`	Allocation instantanée	Liste simplement chaînée
`PENDING`	Équité de traitement (*Round-Robin*)	Liste circulaire
`DONE`	Retrait sélectif par identifiant	Liste doublement chaînée

3.2 Implémentation des fonctions de contrôle (API)

Le processeur interagit avec le sous-système DMA via deux primitives atomiques. L'optimisation de ces fonctions est critique : leur coût en cycles d'horloge détermine le seuil de rentabilité du DMA. Si le traitement logiciel d'une requête dépasse le coût d'un transfert direct par le processeur, l'intégration du contrôleur devient contre-performante.

— `DMA_Job` : Prélève un descripteur libre, initialise les paramètres et le planifie dans la file d'attente matérielle.

— `DMA_Done` : Valide la bonne exécution d'une tâche terminée et restitue le descripteur au pool global.

TABLE 2 – Séquencement logique et valeurs de retour de l'API logicielle

Service	Logique de fonctionnement	Retour (Type : Valeur)
`DMA_Job` *(src, dest, len)*	1. **Lock**(`FREE`) → Extraction du descripteur. 2. **Unlock**(`FREE`). 3. Initialisation des paramètres (*src, dest, taille*). 4. **Lock**(`PENDING`) → Insertion en file. 5. **Unlock**(`PENDING`).	**Succès** : ID (> 0) **Pool saturé** : 0 **Erreur** : -1
`DMA_Done` *(ID)*	1. **Lock**(`DONE`) → Retrait par identifiant (`ID`). 2. **Unlock**(`DONE`). 3. Si trouvé : **Lock**(`FREE`) → Réinsertion. 4. **Unlock**(`FREE`).	**Succès** : 1 (Libéré) **Job en cours** : 0 **Erreur** : -1 (ID invalide) ou (Déjà FREE)

4 Validation et Analyse des Résultats logicielle

Afin de valider la logique de notre interface indépendamment du développement matériel, une plateforme de simulation émulant l'architecture matérielle a été développée en C via la bibliothèque POSIX Threads (`pthreads`). L'environnement simule la concurrence du système selon un modèle *Producteur/Consommateur* autour de trois types d'acteurs :

— **Threads Processeurs (Producteurs) :** Émulent les CPU du SoC. Ils initialisent les données en mémoire et soumettent les requêtes de transfert via la primitive `DMA_Job`.
— **Thread DMA (Consommateur) :** Émule le contrôleur matériel. Il scrute continuellement la liste `PENDING`, exécute la copie physique des données en arrière-plan, puis déplace le descripteur dans la liste `DONE`.
— **Thread Vérificateur :**Valide l'intégrité du transfert par une comparaison (`memcmp`) entre la source et la destination.

4.1 Test 1 : Tests Fonctionnels et Séquentiels

Simulation avec un Thread processeur qui lance une requête, un thread DMA qui la traite et un vérificateur qui vérifie le travail de la DMA. Afin de certifier les critères d'évaluation du cas d'utilisation de base, notre environnement de simulation s'appuie sur les fonctions de vérification suivantes :

— **Intégrité des données :** Le succès matériel des transferts est certifié par une comparaison octet par octet (`memcmp`) entre la source et la cible.
— **Gestion des erreurs et limites :** Le système rejette instantanément toute soumission érronée. De même, si le processeur tente de soumettre un nombre de tâches strictement supérieur à la capacité maximale du pool (`MAX_JOBS`), le système refuse la requête sans provoquer de blocage ni d'erreur mémoire.
— **Séquencement (FIFO) :** La fonction `verifier_ordre(attendu, nb_jobs)` parcourt la liste des requêtes terminées (`r.done`) à l'envers, confirmant que le

contrôleur DMA a bien dépilé et traité les tâches dans leur ordre chronologique de soumission.

— **Transitions d'états :** Les transitions sont validées par des assertions directes sur l'attribut `status` de la structure `Job`, ainsi que par l'analyse des codes de retour stricts de la fonction `DMA_done` (0 pour un job en cours, 1 pour un succès, -1 pour une erreur).
— **Cohérence des ressources :** La fonction `verif_total_jobs()` utilise `count_jobs()` pour recenser les descripteurs dans les différentes listes et s'assurer qu'aucun `Job` n'est perdu durant le cycle de vie de la requête.

4.2 Test 2 : Concurrence et intégrité structurelle

Ce scénario sollicite l'architecture en instanciant de multiples threads (2 et 4 dans nos tests) agissant de manière simultanée. L'objectif est de prouvez la résistance du système face aux accès concurrents massifs. Le protocole a permis de valider les critères suivants :

— **Absence d'interblocage et de compétition :** La manipulation simultanée des listes par plusieurs processeurs ne provoque aucune corruption de pointeurs (absence d'erreur de segmentation). La politique d'acquisition et de relâchement des verrous (*mutex*) est saine : aucun *deadlock* n'est constaté, garantissant une exécution continue sans blocage du programme.
— **Suivi strict des requêtes (Aucune perte) :** Un contrôle continu vérifie qu'aucun descripteur ne disparaît en mémoire, même sous forte charge concurrentielle. À tout instant de l'exécution, le nombre total d'éléments répartis dans les trois listes (`FREE`, `PENDING`, `DONE`), additionné à la tâche potentiellement en cours de traitement par la DMA, reste toujours strictement égal au nombre de descripteurs alloués au démarrage (`MAX_JOBS`).

4.3 Test 3 : Multi-threads et requêtes aléatoires

La simulation déploie de multiples threads générant un flux imprévisible de requêtes, combiné à un thread de surveillance. L'objectif est de tester la fiabilité du système face aux critères. Les critères cités sont validés :

— **Tolérance aux fautes (Robustesse) :** L'injection intentionnelle et aléatoire de requêtes invalides permet de certifier que le système rejette correctement l'opération sans perturber le traitement des requêtes valides parallèles.
— **Surveillance continue de l'intégrité :** verrouillage périodiquement des listes pour s'assurer que la somme totale des descripteurs reste strictement constante. Cela prouve l'absence absolue de perte de descripteurs ou de corruption des chaînages sous forte charge.
— **Résistance à la saturation :** La simulation provoque des états de saturation du pool de requêtes.
— **Cohérence des transferts :** Chaque thread vérifie que les données récupérées après son propre acquittement (`DMA_done`) sont exactes.

Résultat : La simulation demeure parfaitement stable. L'API gère l'épuissement des sources de manière déterministe : les requêtes excédentaires ne provoquent ni attente infinie (*starvation*) ni plantage du système. Les threads concernés reçoivent instantanément le code de retour d'erreur prévu (-1), prouvant que le système est

capable de subir une charge extrême tout en informant correctement le processeur de son indisponibilité.

5 Architecture et Conception Matérielle (VHDL)

L'architecture matérielle du contrôleur DMA est encapsulée dans un composant unique connecté au bus du SoC. Ce module est subdivisé en quatre blocs fonctionnels spécialisés qui communiquent en interne via des registres et des signaux de contrôle dédiés.

5.1 Interconnexions internes et registres de communication

Le processeur configure le contrôleur, tandis que la DMA prend le contrôle du bus pour exécuter les transferts. Les échanges internes entre les trois sous-modules sont orchestrés par les signaux et registres décrits dans la Figure suivante.

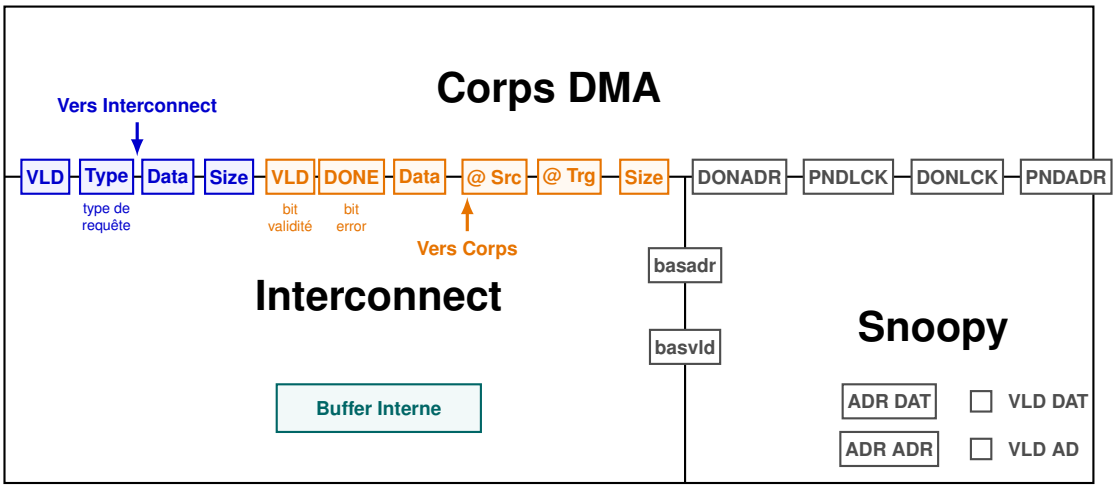


FIGURE 4 – Architecture interne détaillée : Interfaces, structures de stockage et flux de registres

La logique matérielle du bloc *Corps* repose sur un automate fonctionnant en étoile. À partir d'un état d'attente central, la machine à états (FSM) se divise en six branches distinctes. Chaque branche est dédiée à une seule opération du transfert pour que le processus soit simple et direct :

- `get_lock` : Verrouiller la liste pour y accéder en toute sécurité.
- `get_job` : Lire les informations de la tâche (adresses, taille).
- `read` : Lire les données depuis la mémoire source.
- `store` : Écrire les données dans la mémoire de destination.
- `Done` : Mettre à jour le statut pour dire que la tâche est finie.
- `release_lock` : Déverrouiller la liste pour rendre l'accès au processeur.

5.2 Machines à états finis (FSM) du bloc Corps

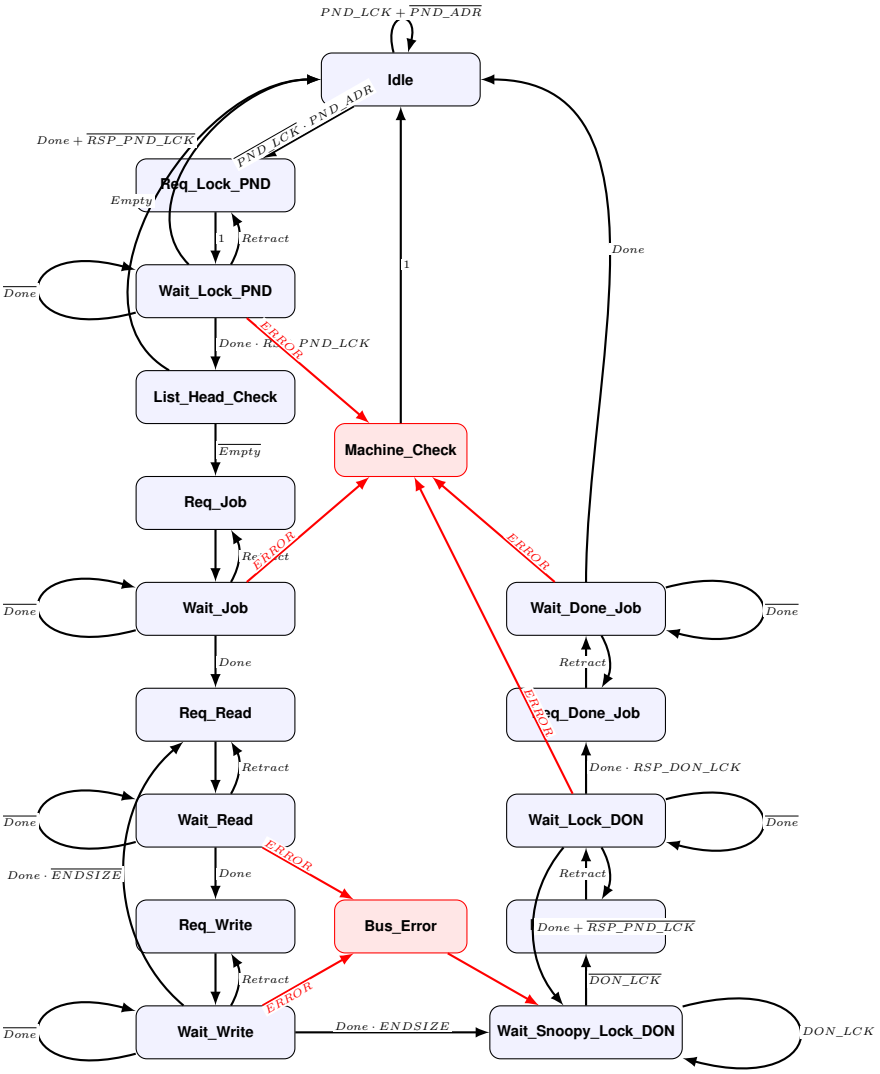


FIGURE 5 – Machine à états finis complète du bloc Corps

5.3 Machines à états finis (FSM) du bloc Interconnect

5.3.1 Début

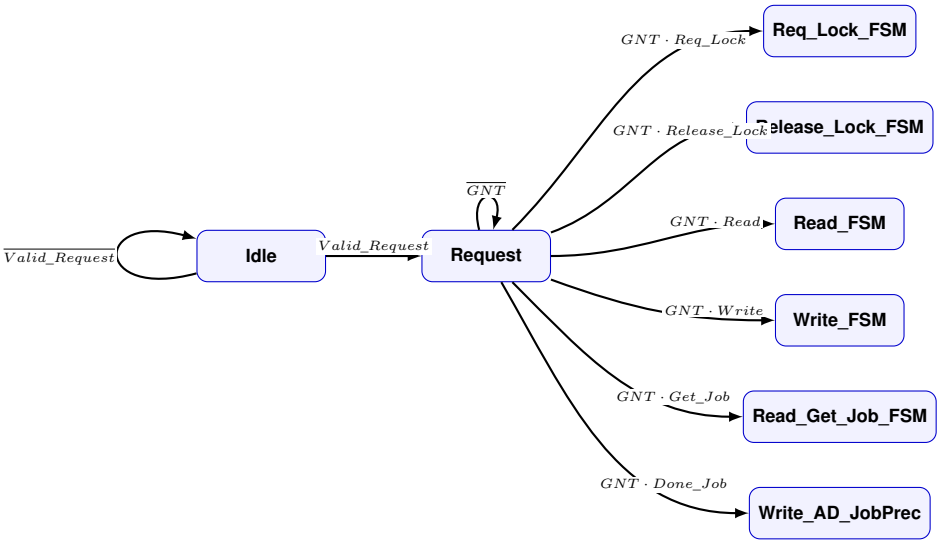


FIGURE 6 – Diagramme d'états : Interconnect Start FSM

5.3.2 Requête Req_Lock

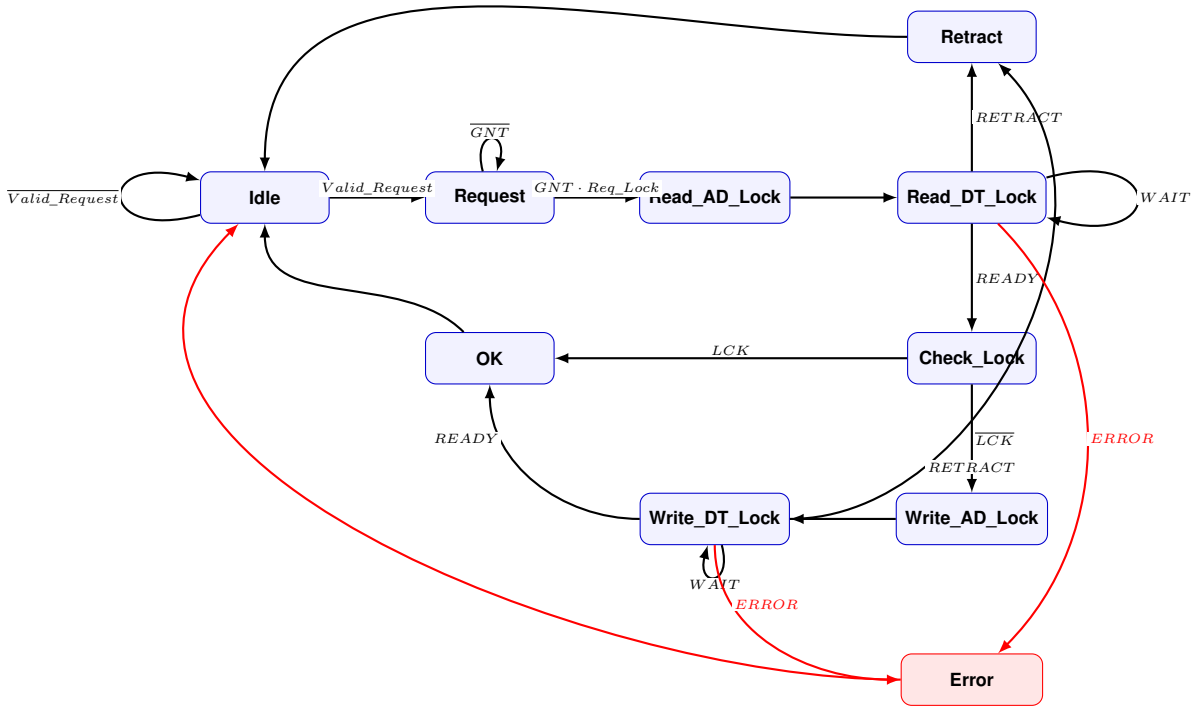


FIGURE 7 – Diagramme d'états : Interconnect Req Lock FSM (Format compact)

5.3.3 Requête Release_lock

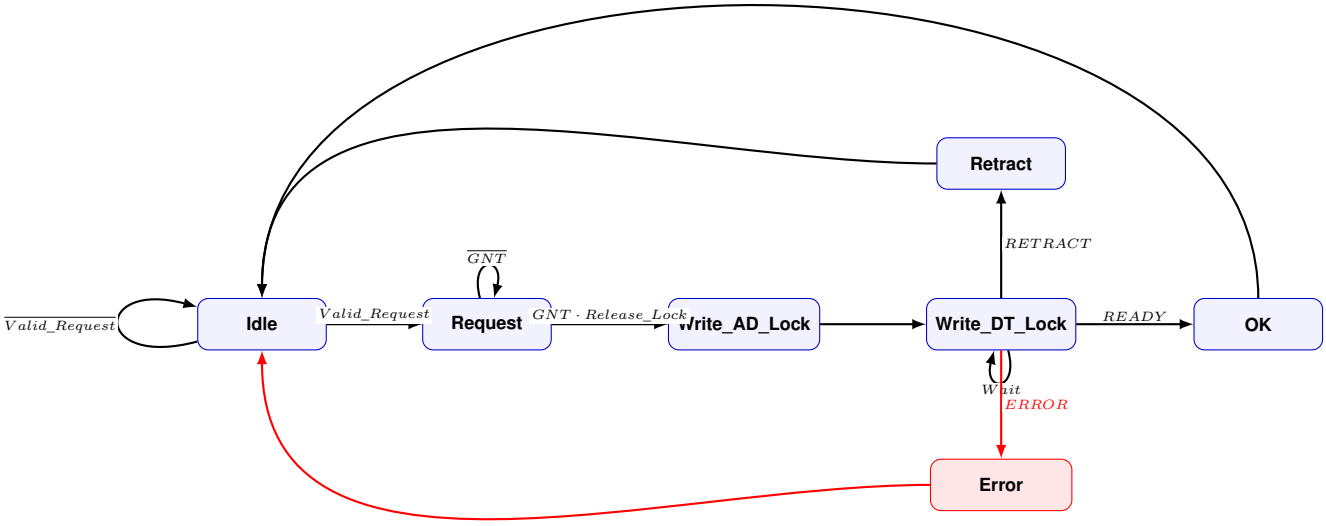


FIGURE 8 – Diagramme d'états : Interconnect Release Lock FSM

5.3.4 Requête Get_Job

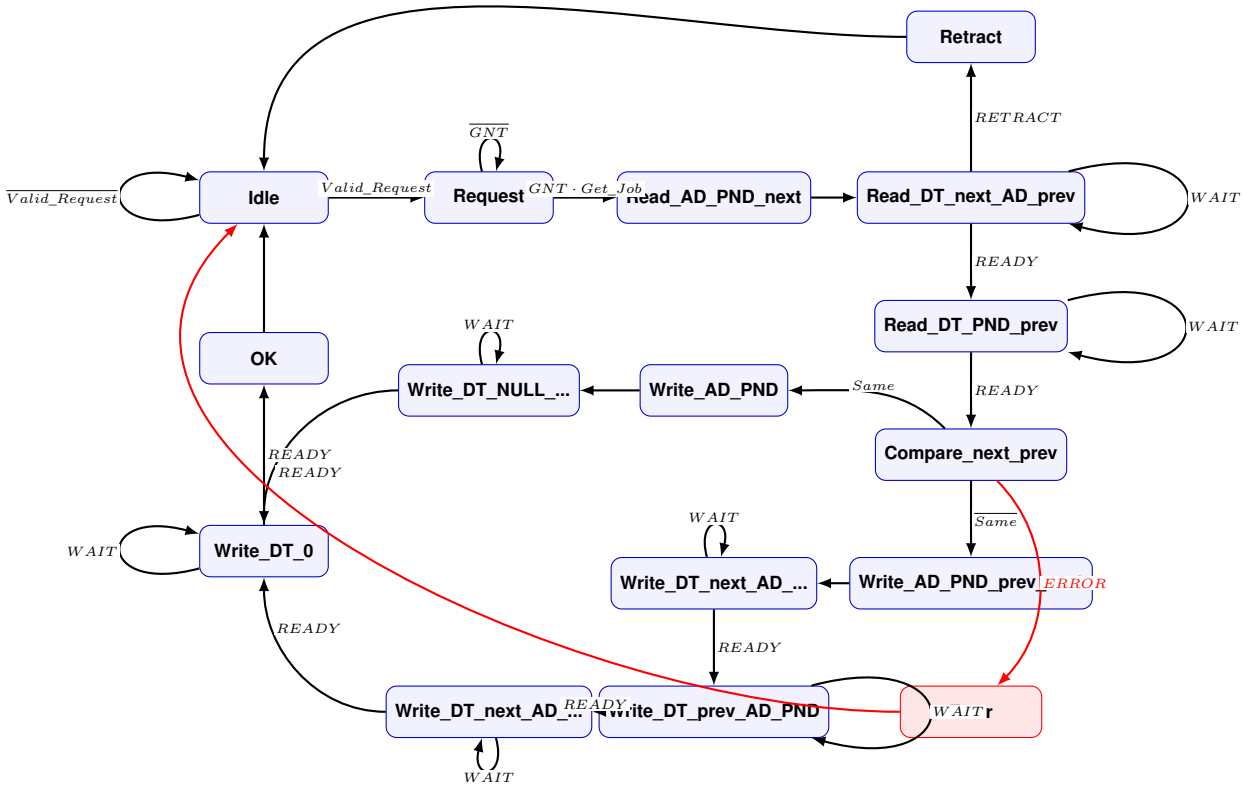


FIGURE 9 – Diagramme d'états : Interconnect Get Job FSM (Format compact)

5.3.5 Requête Read

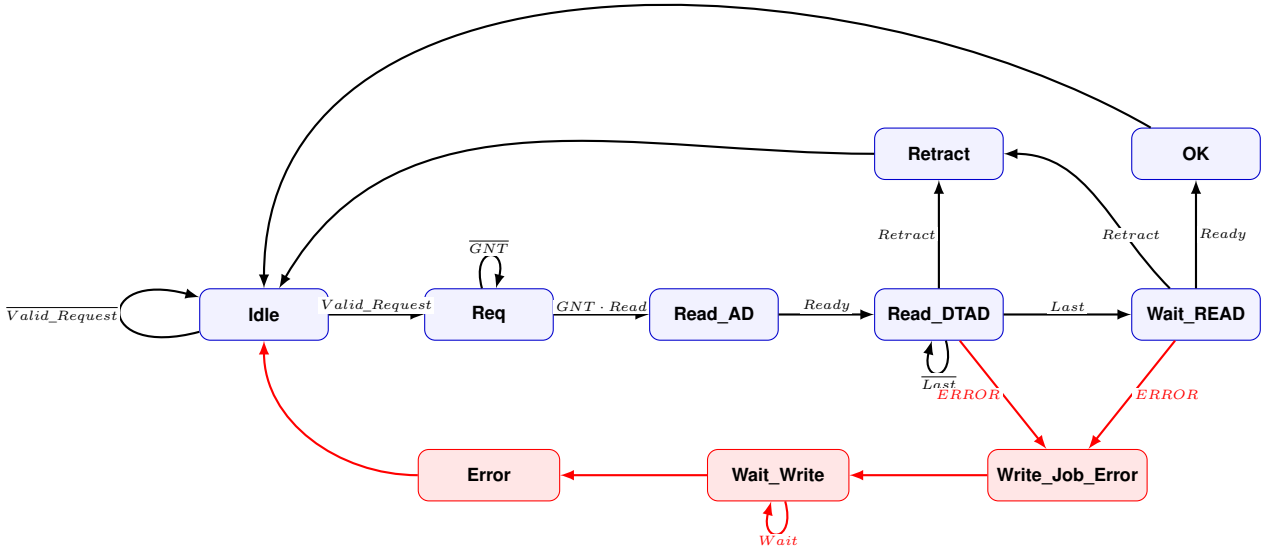


FIGURE 10 – Diagramme d'états de la machine de contrôle de lecture (Interconnect)

5.3.6 Requête Store

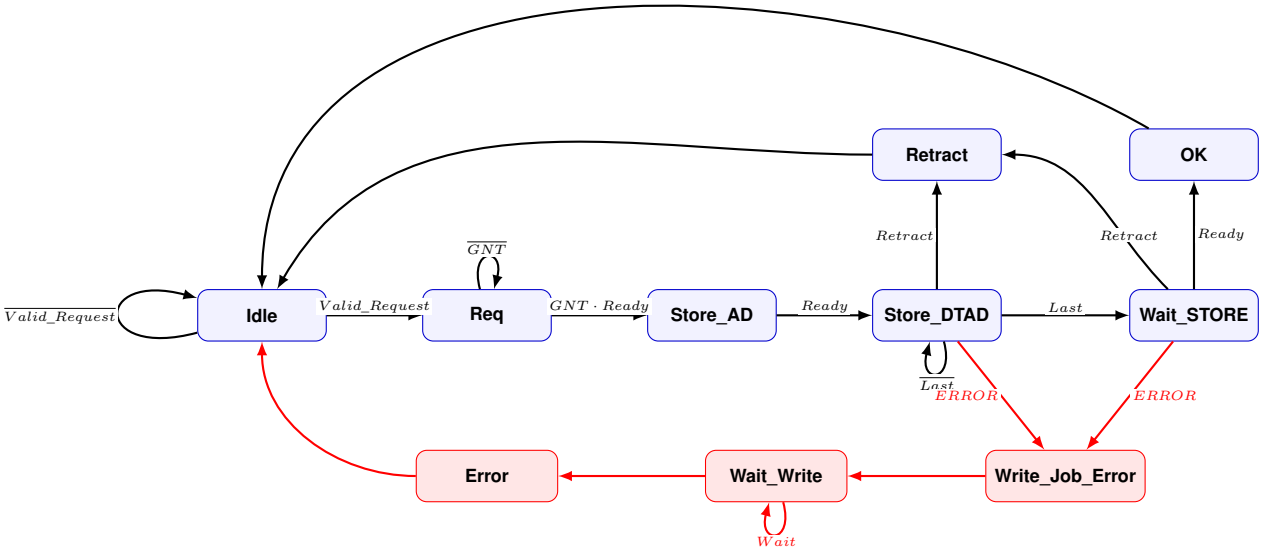


FIGURE 11 – Diagramme d'états de la machine de contrôle d'écriture (Store)

5.3.7 Requête Done_job

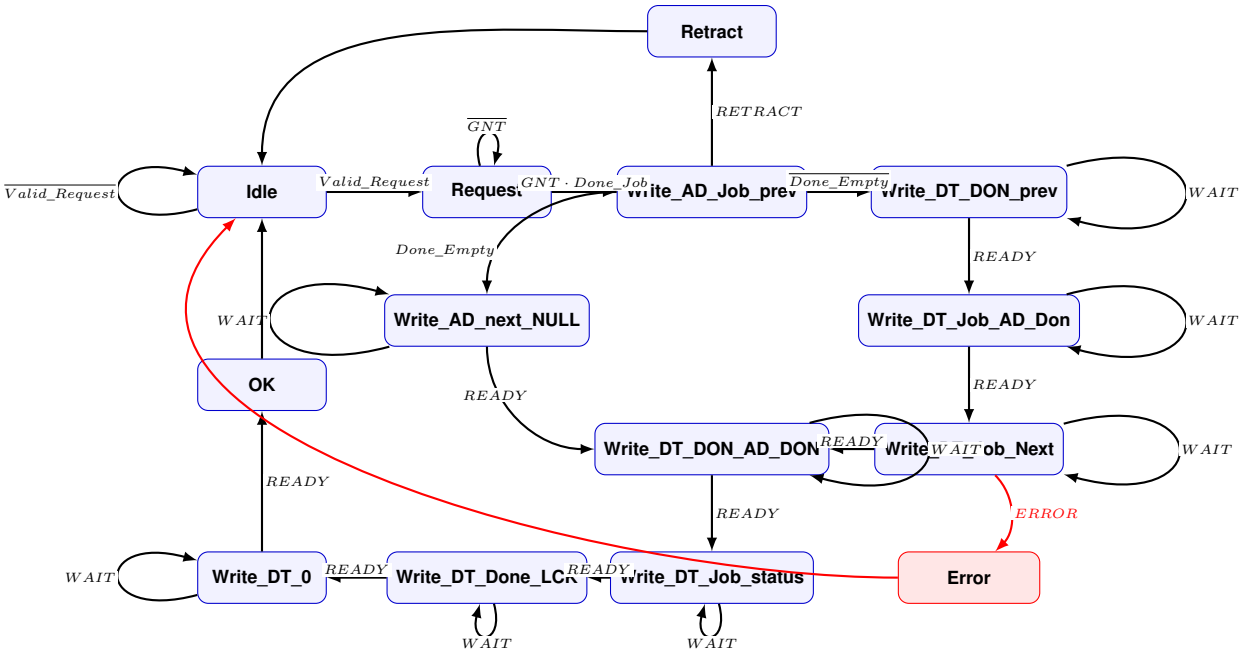


FIGURE 12 – Diagramme d'états : Interconnect Done Job FSM (Format compact)

5.4 Calcul de performance de l'interface logicielle

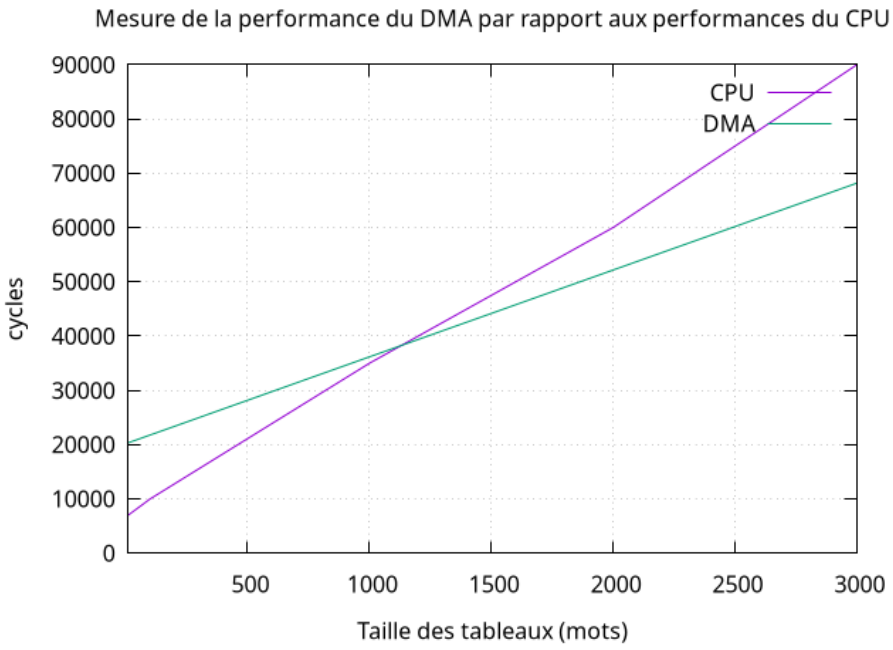
Le calcul de la durée d'exécution des fonctions DMA_job() et DMA_done(), ainsi que du nombre de cycles nécessaires au DMA pour traiter une tâche, ne sert pas

uniquement de statistique générale : il permet d'estimer la taille minimale du bloc mémoire à transférer pour laquelle l'utilisation du DMA devient plus efficace que celle du processeur.

5.4.1 Méthodologie

— On calcule le nombre de cycles du DMA en sommant le nombre d'états traversés au cours du cycle de vie d'une tâche. On suppose pour l'estimation conservatrice que chaque état prend 1 cycle.
— Pour estimer le coût en cycles des fonctions `DMA_job()` et `DMA_done()`, nous utilisons le petit système d'exploitation GIET (Gestionnaire d'Interruptions, Exceptions et Trappes).
— Configuration du cache utilisée pour les mesures et estimations : cache de 64 KB, lignes/mots de 64 bytes, Write-Through, correspondance directe, Snoopy activé.
— Afin d'obtenir le pire cas, le nombre de cycles est mesuré avec le cache froid (cache vide) avant l'exécution.

5.4.2 Résultats



Ces résultats de simulation montrent qu'il est plus avantageux d'utiliser notre DMA pour effectuer des transferts de mémoire lorsque la taille du segment dépasse 1000 mots. Cependant, ces résultats doivent être confirmés par des tests sur un SoC physique afin de déterminer l'efficacité réelle de notre DMA.